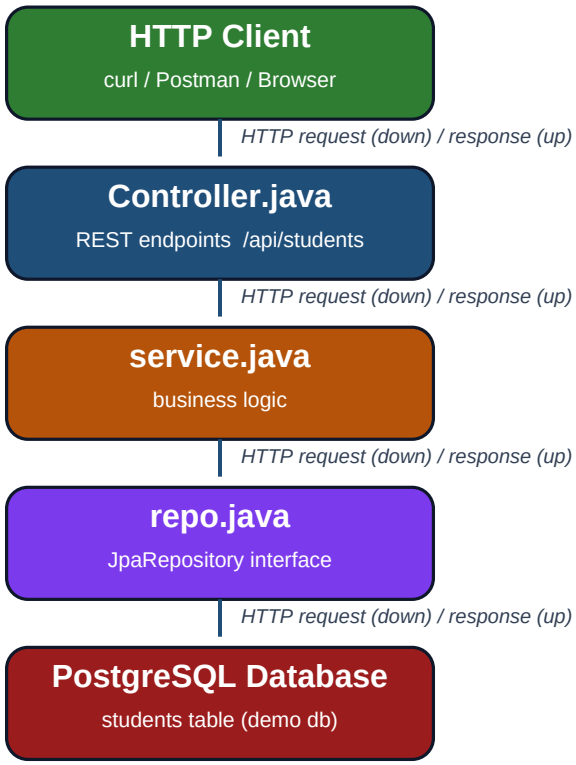
Student REST API - Project Flow

A Spring Boot 4 / JPA / PostgreSQL REST API for managing student records. This document explains the request flow, architecture layers, and each available endpoint.

1. Request Flow Architecture

Every API call follows a strict layered pattern: the request enters through the **Controller** (HTTP layer), moves to the **Service** (business logic layer), then to the **Repo** (data access layer using Spring Data JPA), and finally reaches the **PostgreSQL database**. The response travels back up the same chain to the client.



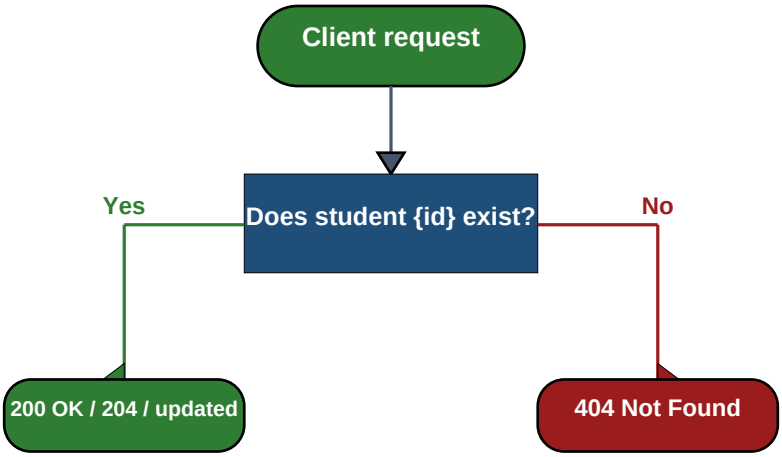
2. Endpoint Decision Flow

The table below describes the exact behaviour of each endpoint, including the HTTP status code returned in both the success and failure scenarios.

Method	URL	Action	Service method	Success	Error
GET	/api/students	List all students	getAllStudents()	200 OK + JSON list	-
GET	/api/students/{id}	Fetch one student	getStudentById(id)	200 OK + student	404 Not Found
POST	/api/students	Create a student	createStudent(s)	201 Created + student	400 if invalid body
PUT	/api/students/{id}	Update a student	updateStudent(id, s)	200 OK + updated	404 Not Found
DELETE	/api/students/{id}	Delete a student	deleteStudent(id)	204 No Content	404 Not Found

3. Lookup / Mutation Decision (e.g. GET /{id}, PUT, DELETE)

In the following flow diagram, the 'Exists?' check is implemented via `findById(id)` (GET/PUT) or `existsById(id)` (DELETE) inside the service layer.



4. Technology Stack & Configuration

Layer	Technology
Language	Java 21
Framework	Spring Boot 4.0 (starter-webmvc, starter-data-jpa)
ORM	Spring Data JPA / Hibernate
Database	PostgreSQL (driver 42.7.12)
Code helpers	Project Lombok (@Getter/@Setter/@NoArgsConstructor)
Build tool	Maven (mvnw wrapper)

Database configuration (src/main/resources/application.yaml)

```
spring.datasource.url: jdbc:postgresql://localhost:5432/demo
spring.datasource.username: postgres
spring.datasource.password: 0000
spring.jpa.hibernate.ddl-auto: update (auto-creates/updates the 'students' table)
spring.jpa.show-sql: true
```

Entity schema (student.java -> table 'students')

Column	Java field	Type	Notes
s_id	s_id	int	Primary key, auto-generated (IDENTITY)
s_name	s_name	varchar	Required (nullable=false)
s_ph	s_ph	varchar	Phone number
s_marks	s_marks	int	Marks obtained

5. Example: End-to-end flow for creating a student (POST)

1. Client sends POST http://localhost:8080/api/students with JSON body { "s_name": "Alice", "s_ph": "9998887777", "s_marks": 95 }.

2. Controller.create(@RequestBody student s) receives the JSON and passes it to the service.

3. service.createStudent(s) resets the id via setS_id(0) so the DB generates a new PK, then calls repo.save(s).

4. repo (JpaRepository) persists the row into the 'students' table in PostgreSQL.

5. Hibernate returns the saved entity (now with the generated s_id).

6. Controller wraps it in ResponseEntity.status(CREATED) and returns 201 Created + the JSON back to the client.

6. How to run the project

1. Make sure PostgreSQL is running locally on port 5432.

2. Create a database named 'demo' (the 'students' table is created automatically).

3. Build: .\mvnw.cmd clean package

4. Run: .\mvnw.cmd spring-boot:run (starts server on http://localhost:8080)

5. Test: GET http://localhost:8080/api/students in Postman, curl, or a browser.